AI FAQ Assistant API

Team No: 12

Team Head: E.SOBIKA

NM ID: (3C617FCDEF4F4928F0140C2E99EBEC7D)

Team Member: A.SALINI

NM ID: (5D1B6176072EA5CAE5F47C26EB55BDCD)

Team Member: K.SHALINI

NM ID: (57768CA5A59825183C74F9664EB7F42A)

Team Member: M.SHALINI

NM ID:(0B7690CA8EABF9687674AB20E1799889**)**

Team Member: M S.SHALINI

NM ID: (F60DCB2A87A985CB36C464C7C824590F)

Description

The AI FAQ Assistant is a robust, secure, and intelligent REST API platform designed to streamline FAQ creation, user authentication, and AI-powered content management. Built with Node.js and Express.js, the application utilizes MongoDB as its primary data store along with Mongoose ODM for elegant data modeling and schema validation.

Security is central to the architecture, implementing JWT (JSON Web Tokens) for stateless user sessions and route guarding alongside bcrypt for strong password encryption. The backend handles intensive operations like automated FAQ generation, semantic search queries, and dynamic context parsing. It also features robust centralized error handling and response status sanitization to prevent unhandled database leakage and ensure operational durability.

Scenario-Based Case Study Background

Priya is a customer support operations manager who handles massive backlogs of repetitive user inquiries but struggles to organize entries, classify topics, write precise summaries, and track client question trends efficiently.

Problem

Difficulty generating and maintaining up-to-date documentation workflows.
Limited visibility into semantic search match keywords and
rapid query resolutions. Time-consuming manual
processes for drafting complex support answers.
Lack of unified database permission layers protecting private documentation from public adjustments.

Solution

The AI FAQ Assistant backend delivers a centralized, secure API wrapper that decouples application business logic from frontend delivery systems. It introduces internal verification matrices controlling specialized services for

content creation, category keywords, database persistence, and automated text inference modules.

Usage

Authenticated Creator Usage: Registers profiles, requests instant AI assistance, drafts official FAQ schemas, and updates or removes selfauthored records.

Public/Consumer Usage: Queries general list routes to consume published articles and issues localized semantic queries via parameters.

Outcome

By offloading workflow controls to a robust backend, operational delays are minimized. Data structural integrity is strictly maintained through automated schema validation, security tokens guarantee that only original authors modify documents, and high-performance indexing facilitates real-time data discoverability.

The AI FAQ Assistant backend delivers a centralized, secure API wrapper that decouples application business logic from frontend delivery systems. It introduces internal verification matrices controlling specialized services for content creation, category keywords, database persistence, and automated text inference modules.

Usage

Authenticated Creator Usage: Registers profiles, requests instant AI assistance, drafts official FAQ schemas, and updates or removes selfauthored records.

Public/Consumer Usage: Queries general list routes to consume published articles and issues localized semantic queries via parameters.

1. Software Requirements

Operating System: Windows 10/11, macOS, or Linux

(supports cross-platform operations). Node.js (v18 or above): Runtime ecosystem running server logic and routing infrastructure. `npm (v9 or above): Node package manager to control operational dependencies.` Express.js (v4/v5 Ecosystem): Lightweight routing web framework to construct backend RESTful entry points. MongoDB / Mongoose ODM: Document NoSQL engine storing application users and curated data entities. Postman / ThunderClient: API verification toolkit to assert schema validation responses across administrative route shields.

Code Editor: Visual Studio Code or similar IDE environment.

2. Hardware Requirements

Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.

RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB processing layers). Storage: 1 GB of available disk workspace.

Epic-1: Project Architecture

The AI FAQ Assistant application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

Components Definition

Interface: API tools like Postman query distinct route paths to extract JSON entities.

Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handles CORS headers.

Authentication Middleware: Extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths.

Granular Services (Logic Modules): Enforces isolated domain rules for user session life cycles and operational query modifications.

Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.

AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex questionanswering tasks and automated topic generation workflows.

Entity-Relationship (ER) Diagram Description

The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

1. User Entity

_id (Primary Key -

ObjectId) name (String,

Required) email (String,

Required,

Unique) password

(String, Hashed,

Required) role (String,

Default: "user")

createdAt

updatedAt

2. FAQ Entity

_id (Primary Key -

ObjectId) question

(String, Required)

answer (String,

Required) category (String,

Required)

createdBy (Foreign Key referencing

User Collection) createdAt updatedAt

3. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI

before persistence.) topic (String)

generatedQuestion (String)

Entity-Relationship (ER) Diagram Description

The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

4. User Entity

_id (Primary Key -

ObjectId) name (String,

Required) email (String,

Required,

Unique) password

(String, Hashed,

Required) role (String,

Default: "user")

createdAt

updatedAt

5. **FAQ Entity**

_id (Primary Key - ObjectId) question (String, Required) answer (String, Required) category (String, Required)

createdBy (Foreign Key referencing User Collection) createdAt
updatedAt

6. **AI Generation Entity (Logical)**

(Generated dynamically using Google Gemini AI before persistence.) topic (String)

generatedQuestion (String)

generatedAnswer (String) generatedCategory (String) generatedAt

Entity-Relationship (ER) Diagram Description

The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

7. **User Entity**

_id (Primary Key -

ObjectId) name (String,

Required) email (String,

Required,

Unique) password

(String, Hashed,

Required) role (String,

Default: "user")

createdAt

updatedAt

8. FAQ Entity

_id (Primary Key -

ObjectId) question

(String, Required)

answer (String,

Required) category (String,

Required) createdBy (Foreign Key

referencing

User Collection) createdAt updatedAt

9. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI

before persistence.) topic (String)

generatedQuestion

(String)

generatedAnswer

(String) generatedCategory

(String) generatedAt

10. **Category Entity (Logical)** categoryName

(String) description

(String) totalFAQs

(Number)

Key Structural Mappings

User FAQ (One-to-Many): A registered user can create and manage multiple FAQ records. Category FAQ (One-to-Many):

Each category can contain multiple FAQ documents.

User AI Generated FAQ (One-to-Many): A user can generate multiple AIassisted FAQs, which may later be saved into the FAQ collection.

FAQ Search Results (Logical Relationship): Search queries retrieve matching FAQ documents using keyword and semantic matching.

11. **User Entity**

_id (Primary Key -

ObjectId) name (String,

Required) email (String,

Required,

Unique) password

(String, Hashed,

Required) role (String,

Default: "user")

createdAt

updatedAt

12. FAQ Entity

_id (Primary Key -

ObjectId) question

(String, Required)

answer (String,

Required) category (String,

Required)

createdBy (Foreign Key referencing

User Collection) createdAt
updatedAt

13. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI

before persistence.) topic (String)

generatedQuestion

(String)

generatedAnswer

(String) generatedCategory

(String) generatedAt

14. **Category Entity (Logical)** categoryName

(String) description

(String) totalFAQs

(Number)

Key Structural Mappings

User FAQ (One-to-Many): A registered user can create and manage multiple FAQ records. Category FAQ (One-to-Many): Each category can contain multiple FAQ documents.

User AI Generated FAQ (One-to-Many): A user can generate multiple AIassisted FAQs, which may later be saved into the FAQ collection.

FAQ Search Results (Logical Relationship): Search queries retrieve matching FAQ documents using keyword and semantic matching.

Entity-Relationship (ER) Diagram Description

The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

15. **User Entity**

_id (Primary Key - ObjectId) name (String, Required) email (String, Required, Unique) password (String, Hashed, Required) role (String, Default: "user") createdAt

updatedAt

16. FAQ Entity

_id (Primary Key - ObjectId) question (String, Required) answer (String, Required) category (String, Required) createdBy (Foreign Key referencing User Collection) createdAt updatedAt

17. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI before persistence.) topic (String) generatedQuestion (String) generatedAnswer (String) generatedCategory

(String) generatedAt

18. **Category Entity (Logical)** categoryName

(String) description

(String) totalFAQs

(Number)

Key Structural Mappings

User FAQ (One-to-Many): A registered user can create and manage

multiple FAQ records. Category FAQ (One-to-Many):

Each category can contain multiple FAQ documents.

User AI Generated FAQ (One-to-Many): A user can generate multiple AIassisted FAQs, which may later be saved into the FAQ collection.

FAQ Search Results (Logical Relationship): Search queries retrieve matching FAQ documents using keyword and semantic matching.

Key Features

JWT Authentication & Authorization: Secure token-based authentication protects private API endpoints.

AI FAQ Generation: Google Gemini AI automatically generates questions,

answers, and topic suggestions. Semantic FAQ Search: Intelligent

keywordbased search enables faster information retrieval.

Complete CRUD Operations: Create, Read, Update, and Delete FAQ records. Category Management: Organizes FAQs into structured categories

for easier navigation. Password Encryption: User passwords are

securely hashed using bcrypt before storage. Centralized Error

Handling: Standardized API error responses improve reliability and

debugging. Schema Validation: Mongoose validation ensures data

integrity before database persistence. RESTful API Architecture:

Modular Express.js routes provide scalable backend services.

MongoDB Integration: Efficient NoSQL document storage using

Mongoose ODM.

Roles and Responsibilities

Admin: Manage all registered users; view, update, and delete any FAQ; monitor AI-generated content; manage FAQ categories; access complete system functionality.

Content Creator: Create new FAQs; edit personal FAQ records; delete owned FAQs; generate FAQs using AI; organize FAQs under categories.

Authenticated User: View public FAQs; search FAQs using keywords; generate AI-assisted answers; access personal profile information.

Public User: View published FAQs; search FAQs; cannot create, edit, or delete FAQ records; no access to protected API endpoints.

User Flow MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components.

Model Layer (Data Blueprint): Defines structural blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets field rules and defaults before validation persistence occurs.

Controller Layer (Execution Matrix): The intermediary brain of the ecosystem. It captures request vectors from execution routes, checks structural request constraints, delegates processing workflows directly to models or services, and packages resultant output into standard JSON return arrays.

View Layer (Routing Network Interface): In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

Epic 2: Project Setup and Configuration Creating Project Folder

Create a new folder with your . Inside

that folder, create a new subfolder

named src. Now open that folder in

VS Code

Server Setup (npm init)

```
Open your root folder terminal in VS Code and run: npm
 init -y npm install express mongoose dotenv cors
 jsonwebtoken bcrypt @google/genai
```

File and Folder Structure Matrix Epic 3: Backend Development Core System Script Implementations 1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

2. Gateway Express Setup (src/app.js)

3. Security Access Middleware (src/middleware/authMiddleware.js)

Intercepts incoming target queries, extracts transmission header authorization records, and validates token identity context claims:

4. Centralized Error Handler Middleware

(src/middleware/errorMiddleware.js) Epic 4: Database Configuration

Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

API Router Interface Endpoints 1. Public Authentication Operations (src/routes/authRoutes.js)

2. Protected FAQ Operations (src/routes/faqRoutes.js)

JavaScript

3. AI Automation Operations

(src/routes/aiRoutes.js) Epic 5: Project Executions Running the API Application

Install Postman or ThunderClient (VS Code extension).

Start the backend development server environment parameters:

```
npm run dev
```

API Testing Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

1. Identity Onboarding Path Registration

```
Route: POST
/api/auth/register
Access: Public
```

Payload Request Body JSON:

```
{
"name": "Priya
Sharma", "email":
"priya@writeflow.c om",
 "password":
"securepassword123
"
}
```

2. Profile Login Verification Query

```
Route: POST
/api/auth/login Access:
Public
```

Payload Request Body JSON:

```
{
"email":
"priya@writeflow.com",
"password":
"securepassword123"
}
```

3. Resource Entity Creation (FAQ)

```
Route: POST /api/faqs
Access: Private (Bearer <JWT Token>) Payload
```

Request Body JSON:

```
{
"question": "How do I configure custom category tags?",
"answer": "Navigate to settings panel, append keyword
 tags arrays, and save schema.",
 "category": "Configuration"
 }
Route: POST
/api/auth/register
Access: Public
```

Payload Request Body JSON:

```
 {
 "name": "Priya
```

```
Sharma", "email":
"priya@writeflow.c om",
 "password":
"securepassword123
"
}
```

4. Profile Login Verification Query

```
Route: POST
/api/auth/login Access:
Public
```

Payload Request Body JSON:

```
{
"email":
"priya@writeflow.com",
"password":
"securepassword123"
}
```

5. Resource Entity Creation (FAQ)

```
Route: POST /api/faqs
Access: Private (Bearer <JWT Token>) Payload
```

Request Body JSON:

```
{
"question": "How do I configure custom category tags?",
"answer": "Navigate to settings panel, append keyword
tags arrays, and save schema.",
"category": "Configuration"
}
```

6. Keyword Resource Entity Fetching (Search)

```
Route: GET
/api/faqs/search?q=configu
re Access: Public
```

7. AI Intelligent Content Automation (Generate FAQ Pair)

```
Route: POST
/api/ai/generate-faq Access:
Private
(Bearer <JWT Token>)
```

Payload Request Body JSON:

```
{
"topic": "Mongoose schema indexing validation runtime
workflow optimization"
}
```

```
Route: POST
/api/auth/register
Access: Public
```

Payload Request Body JSON:

```
{
"name": "Priya
Sharma", "email":
"priya@writeflow.c om",
 "password":
"securepassword123
"
}
```

Route: POST
/api/auth/register
Access: Public

Payload Request Body JSON:

```
{
"name": "Priya
Sharma", "email":
"priya@writeflow.c om",
 "password":
"securepassword123
"
}
```

8. Profile Login Verification Query

Route: POST
/api/auth/login Access:
Public

Payload Request Body JSON:

```
{
"email":
"priya@writeflow.com",
"password":
"securepassword123"
}
```

9. Resource Entity Creation (FAQ)

Route: POST /api/faqs
Access: Private (Bearer <JWT Token>) Payload

Request Body JSON:

```
{
"question": "How do I configure custom category tags?",
"answer": "Navigate to settings panel, append keyword
tags arrays, and save schema.",
"category": "Configuration"
}
```

10. Keyword Resource Entity Fetching (Search)

```
Route: GET
/api/faqs/search?q=configu
re Access: Public
```

11. AI Intelligent Content Automation (Generate FAQ Pair)

```
Route: POST
/api/ai/generate-faq
Access: Private
(Bearer <JWT Token>)
```

Payload Request Body JSON:

```
{
"topic": "Mongoose schema indexing validation runtime
workflow optimization"
}
```